PYTHON ANALYSIS PROGRAMS

Complete Programs for Practice and Lab Submission

1. Student Marks Analysis

```
students = [
    ("Arun", (78, 85, 92, 67)),
    ("Bala", (45, 67, 55, 49)),
    ("Charan", (90, 92, 88, 95)),
    ("Divya", (65, 72, 70, 68)),
    ("Esha", (88, 76, 91, 84))
]

totals = {}
averages = {}

print("Student Totals:")
for name, marks in students:
    total = sum(marks)
    avg = total / len(marks)
    totals[name] = total
    averages[name] = avg
    print(name + ":", total)

print("\nStudent Averages:")
for name in averages:
    print(name + ":", format(averages[name], ".2f"))

print("\nHighest Total:", max(totals, key=totals.get))
print("Lowest Average:", min(averages, key=averages.get))

class_average = sum(averages.values()) / len(averages)
print("\nAbove Class Average:")
for name in averages:
    if averages[name] > class_average:
        print(name)

print("\nStudents with 3 or more marks above 80:")
for name, marks in students:
    if sum(mark > 80 for mark in marks) >= 3:
        print(name)

print("\nSubject-wise Highest:")
for i in range(4):
    print("Subject", i + 1, ":", max(s[1][i] for s in students))

print("\nStudents with a mark below 50:")
for name, marks in students:
    if any(mark < 50 for mark in marks):
        print(name)

sorted_students = sorted(averages.items(), key=lambda x: x[1], reverse=True)
print("\nStudents Sorted by Average (Descending):")
for name, avg in sorted_students:
    print(name, "-", format(avg, ".2f"))

print("\nTop 2:")
for name, avg in sorted_students[:2]:
    print(name)

print("\nMutable and Immutable:")
print("List 'students' is Mutable.")
print("Tuples containing student name and marks are Immutable.")
```

2. Customer Name Analysis

```
names = [
    "Arun Kumar ", "ARUN KUMAR", " arun kumar",
    "Bala Krishnan", "BALA KRISHNAN ",
    "Charan", " charan ", "DIVYA DEVI", "Divya Devi"
]

normalized = [name.strip().lower() for name in names]

print("Normalized Names:")
for name in normalized:
    print(name)

duplicates = []
print("\nDuplicate Customers:")
for name in normalized:
    if normalized.count(name) > 1 and name not in duplicates:
        duplicates.append(name)
        print(name)

unique = list(set(normalized))
print("\nNumber of Unique Customers:", len(unique))

print("\nNames with Multiple Words:")
for name in sorted(unique):
    if len(name.split()) > 1:
        print(name)

print("\nLongest Name:")
print(max(unique, key=len))

print("\nStarts with A or D:")
for name in sorted(unique):
    if name.startswith("a") or name.startswith("d"):
        print(name)

print("\nCharacter Count:")
for name in sorted(unique):
    print(name, ":", len(name))

print("\nUnique Names Alphabetically:")
for name in sorted(unique):
    print(name)

print("\nMost Frequent Customer:")
print(max(unique, key=normalized.count))
```

3. Supermarket Inventory Analysis

```
products = [
    ("Laptop", 5, 55000), ("Mouse", 25, 700),
    ("Keyboard", 8, 1200), ("Monitor", 12, 15000),
    ("Printer", 4, 12000), ("Headset", 15, 2500)
]

print("Inventory Values:")
for name, quantity, price in products:
    print(name, ":", quantity * price)

total = sum(quantity * price for name, quantity, price in products)
print("\nTotal Inventory Value:", total)

print("\nLow Stock:")
for name, quantity, price in products:
    if quantity < 10:
        print(name)

print("\nPrice > 10000:")
for name, quantity, price in products:
    if price > 10000:
        print(name)

print("\nMost Expensive:")
print(max(products, key=lambda x: x[2])[0])

print("\nHighest Inventory Value:")
print(max(products, key=lambda x: x[1] * x[2])[0])

print("\nProducts Sorted by Price:")
for product in sorted(products, key=lambda x: x[2]):
    print(product)

print("\nProducts Sorted by Quantity:")
for product in sorted(products, key=lambda x: x[1]):
    print(product)

print("\nTotal Quantity:", sum(quantity for name, quantity, price in products))

print("\nInventory Value > 50000:")
for name, quantity, price in products:
    if quantity * price > 50000:
        print(name)
```

4. Feedback Text Analysis

```
feedback = """
Python programming is easy to learn.
Python programming is powerful.
Learning Python requires practice.
Practice makes programming skills better.
"""

import string

text = feedback.lower()
text = text.translate(str.maketrans("", "", string.punctuation))
words = text.split()
unique = set(words)

print("Total Words:", len(words))
print("Unique Words:", len(unique))
print("Frequency of python:", words.count("python"))
print("Frequency of programming:", words.count("programming"))

print("\nRepeated Words:")
seen = []
for word in words:
    if words.count(word) > 1 and word not in seen:
        seen.append(word)
        print(word)

print("\nLongest Word:")
print(max(words, key=len))

print("\nUnique Words Alphabetically:")
for word in sorted(unique):
    print(word)

print("\nWords containing 'p':")
for word in sorted(unique):
    if "p" in word:
        print(word)

percentage = len(unique) / len(words) * 100
print("\nPercentage of Unique Words:", round(percentage, 2), "%")
```

5. Employee Skills Analysis

```
employees = [
    ("E101", "Arun", ["Python", "SQL", "ML"]),
    ("E102", "Bala", ["Java", "SQL", "HTML"]),
    ("E103", "Charan", ["Python", "ML", "SQL"]),
    ("E104", "Divya", ["Python", "Java", "Cloud"]),
    ("E105", "Esha", ["Python", "ML", "Cloud"])
]

def show(title, condition):
    print("\n" + title)
    for _, name, skills in employees:
        if condition(skills):
            print(name)

show("Python + ML:", lambda s: "Python" in s and "ML" in s)
show("SQL but not Java:", lambda s: "SQL" in s and "Java" not in s)
show("Java or Cloud:", lambda s: "Java" in s or "Cloud" in s)
show("Employees with 3 skills:", lambda s: len(s) >= 3)
show("Without Python:", lambda s: "Python" not in s)

all_skills = [skill for _, _, skills in employees for skill in skills]
print("\nMost Common Skill:")
print(max(set(all_skills), key=all_skills.count))

show("Suitable for Python + ML:", lambda s: "Python" in s and "ML" in s)
```

6. Bank Transaction Analysis

```
transactions = [
    ("T101", "Deposit", 5000),
    ("T102", "Withdraw", 2000),
    ("T103", "Deposit", 8000),
    ("T104", "Withdraw", 1500),
    ("T105", "Deposit", 3000),
    ("T106", "Withdraw", 7000)
]

total_deposit = sum(amount for _, t, amount in transactions if t == "Deposit")
total_withdraw = sum(amount for _, t, amount in transactions if t == "Withdraw")

print("Total Deposit:", total_deposit)
print("Total Withdrawal:", total_withdraw)
print("Net Balance:", total_deposit - total_withdraw)
print("Largest Deposit:", max(amount for _, t, amount in transactions if t == "Deposit"))
print("Largest Withdrawal:", max(amount for _, t, amount in transactions if t == "Withdraw"))

print("\nTransactions > 4000:")
for tid, _, amount in transactions:
    if amount > 4000:
        print(tid)

deposits = sum(t == "Deposit" for _, t, _ in transactions)
withdrawals = sum(t == "Withdraw" for _, t, _ in transactions)
print("\nDeposits:", deposits)
print("Withdrawals:", withdrawals)

average = sum(amount for _, _, amount in transactions) / len(transactions)
print("Average Transaction:", round(average, 2))

print("\nWithdrawals > 50% of Deposits:")
print("Yes" if total_withdraw > total_deposit * 0.5 else "No")
```

7. Password Analysis

```
passwords = [
    "Python@123", "python123", "PYTHON@123",
    "Py@45", "Python123", "Java#4567", "Admin@2026"
]

valid = []
invalid = []

for p in passwords:
    errors = []
    if len(p) < 8:
        errors.append("Less than 8 characters")
    if not any(c.isupper() for c in p):
        errors.append("No uppercase")
    if not any(c.islower() for c in p):
        errors.append("No lowercase")
    if not any(c.isdigit() for c in p):
        errors.append("No digit")
    if not any(not c.isalnum() for c in p):
        errors.append("No special character")

    if errors:
        invalid.append((p, errors))
    else:
        valid.append(p)

print("Valid:")
for p in valid:
    print(p)

print("\nInvalid:")
for p, errors in invalid:
    print(p, "→", ", ".join(errors))

print("\nValid Count:", len(valid))
print("Invalid Count:", len(invalid))

print("\nLongest Password:")
print(max(passwords, key=len))

print("\nPython-related passwords:")
for p in passwords:
    if "python" in p.lower():
        print(p)

print("\nPasswords sorted by length:")
for p in sorted(passwords, key=len):
    print(p)

print("\nValid Percentage: {:.2f}%".format(len(valid) / len(passwords) * 100))
```

8. Unique Employee Records Analysis

```
records = [
    ("Arun", "Python"),
    ("ARUN", "Python"),
    (" Bala", "Java"),
    ("bala", "Java"),
    ("Charan", "Python")
]

names = []

for record in records:
    name = record[0].strip().lower()
    if name not in names:
        names.append(name)

print(names)

# Unique employees and their skills
employees = {}
for record in records:
    name = record[0].strip().lower()
    skill = record[1]
    employees[name] = skill

print(employees)

# record[0] = "arun" is not allowed because record is a tuple.
# Tuples are immutable and do not support item assignment.
```